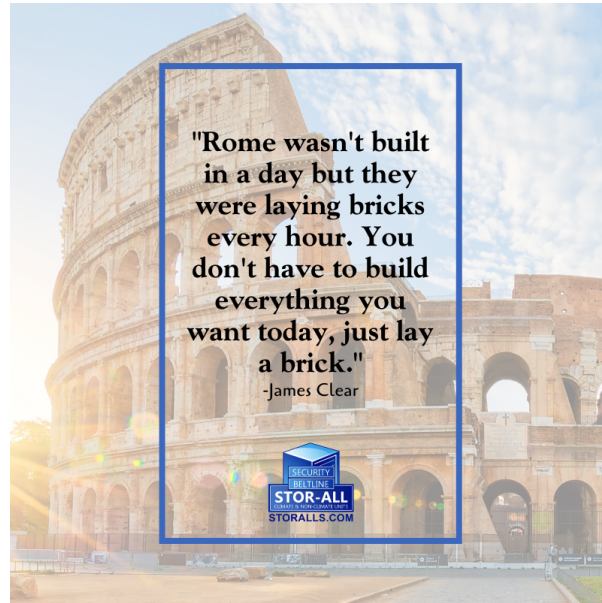


HASHMAP



Good
Morning

Today's Agenda

- Hashmap Introduction
- Frequency of each query
- Count distinct elements
- check if pair sum = k
- Count pair sum = k

Hashmap Implementation → DSA.3



Hotel

5 rooms



Manual register →

Room No.

Occupied

1

✓

2

x

3

✓

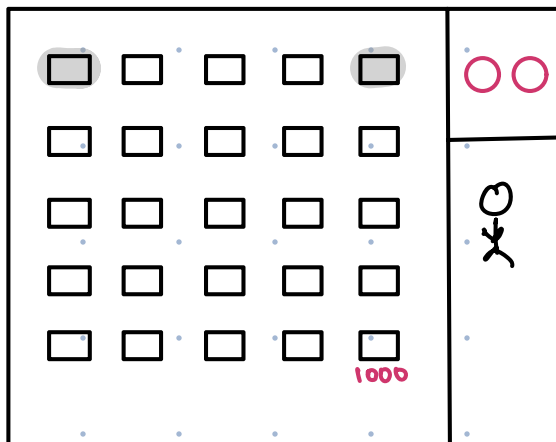
4

✓

5

x

100 rooms



Room no. → {1, 100}

`boolean[] check = new boolean[101]`

True → room is occupied

false → room is free

`check[] =`

X	T	F	F	T	F	F	F	F	F	T
0	1	2	3	4	.	.	.	.	99	100

To check if a particular room is available → $O(1)$

Pandemic → Footfall in hotel has drastically reduced

Manager went to numerologist

→ {1 to 10^9 }

100 rooms

1	4	10	100	172	○○
☐	☐	☐	☐	☐	
200	203	10^7	10^6	999	O X
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
				1000	

One to One mapping

boolean[] check = $10^9 + 1$

To have the information of 100 rooms, we will use memory size of $10^9 + 1$

Lot of memory wastage

Issues

01. Lot of memory wastage
02. Can't create an array of size $> 10^6$

HashMap → A datastructure which holds the information in the form of key-value pair

HashMap < key, value >

Room no.

True/False

HM

$\left[\begin{array}{l} 1 \rightarrow \text{True} \\ 10 \rightarrow \text{false} \\ 100 \rightarrow \text{false} \\ \vdots \\ 10^9 \rightarrow \text{false} \end{array} \right]$

max size of
HM = 100

if ($\text{hm.get}(1) == \text{false}$)

1^{st} room is available &
can be given to customer

TC: $O(1)$

Properties of HashMap

01. Keys must be unique.
02. Values can be anything
03. One null key is allowed.
04. HM doesn't maintain the order of insertion.



1. Population of every country

🇮🇳 India → 145

🇺🇸 USA → 65

🇷🇺 Russia → 14

🇨🇳 China → 147

🇬🇧 United Kingdom → 80

HashMap < key, value >

Country Name

String

population

Long

2. Number of states of every country

🇮🇳 India → 29

🇺🇸 USA → 50

🇨🇳 China → 25

🇷🇺 Russia → 21

```
HashMap < String, Integer > hm = new HashMap < > ();
```

Country Name

No. of states

3. Name of all the states of every country

 India → Andhra Pradesh , Arunachal Pradesh , U.P , M.P , Karnataka ,....

 USA → New York , Washington , Texas ,...

 Russia → Moscow , Kazan , Samara ,...

 China → X , Y , Z ,...

```
HashMap<String, List<String>> hm = new HashMap<>();
```

Country Name

List of state
name

4. Population of each state in each country

 India

Karnataka → 50

Maharashtra → 72

Himachal Pradesh → 36

 USA

Texas → 18

Florida → 16

Washington → 32

 Russia

Kazan $\rightarrow$ 3

Samara $\rightarrow$ 5

Moscow $\rightarrow$ 7

population of each state $\rightarrow$ HM $\langle$ String, long $\rangle$

For every country,

HM $\langle$ String, HM $\langle$ String, long $\rangle$ $\rangle$ hm = new HashMap $\langle$ X $\rangle$;

Country Name

Population of every
state

HashSet

$\hookrightarrow$ To store unique keys

HashSet $\langle$ Keytype $\rangle$ hs = new HashSet $\langle$ $\rangle$;

HM & HS has every operation as constant $O(1)$

HashMap

`HashMap<keytype, valuetype> hm = new HashMap<>();`

01. `hm.size()`; $\rightarrow$ No. of keys in HM
02. `hm.put(key, value)` $\rightarrow$ insert key with value
03. `hm.containsKey(key)` $\rightarrow$ search if key is present & return true or false
04. `hm.get(key)` $\rightarrow$ To get value of key
05. `hm.remove(key)` $\rightarrow$ remove entire entry from HM
06. updation $\rightarrow$ `hm.put(key, newValue);`
07. `hm.keySet()` $\rightarrow$ To iterate on HM & get keys
08. `hm.values()` $\rightarrow$ To iterate on HM & get values

$O(n)$

HashSet

`HashSet<keytype> hs = new HashSet<>();`

01. `hs.size()`; $\rightarrow$ To get hashset size
02. `hs.add(key)`; $\rightarrow$ To add key in hs
03. `hs.contains(key)` $\rightarrow$ To search key in hs
04. `hs.remove(key)` $\rightarrow$ To remove a key
05. `hs.keySet()` $\rightarrow$ To iterate on all keys $\rightarrow O(n)$

< Question > : Given N elements & Q queries. Find the frequency of elements provided in the query.

N = 11

arr → [2 6 3 8 2 8 2 3 8 10 6]
 . . 0 . 1 2 . 3 . 4 5 . 6 . 7 8 . 9 . 10

 $(1 \leq N \leq 10^6)$ $(1 \leq Q \leq 10^5)$

$Q = 4$



 **BF Idea**

2 → 3

8 → 3

3 → 2

5 → ○

For every query, iterate on array & get its count

$$T_C: O(Q * N)$$
$$Sc : O(1)$$

Idea 2. $\rightarrow$ Use hashmap to get frequency of every ele

key $\rightarrow$ Distinct ele

Value $\rightarrow$ Frequency

arr → [2 6 3 8 2 8 2 3 8 10 6]

0 1 2 3 4 5 6 7 8 9 10

HM

Questions

$$\left[\begin{array}{ccc} 2 & \rightarrow & 3 \\ 6 & \rightarrow & 6 \\ 3 & \rightarrow & 2 \\ 8 & \rightarrow & 3 \\ 10 & \rightarrow & 1 \end{array} \right]$$

2 $\rightarrow$  hm.get(2) $\Rightarrow$ 3

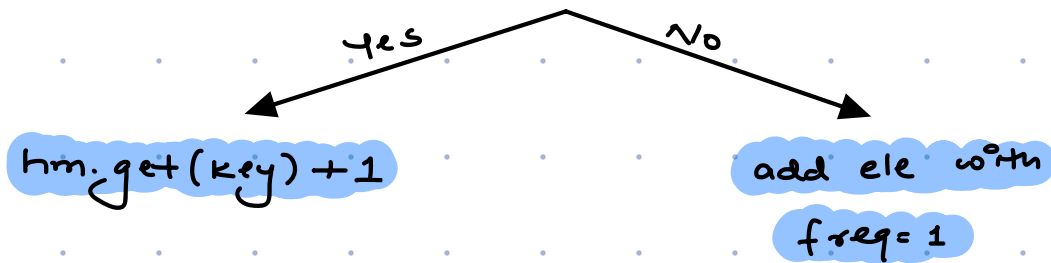
8 $\rightarrow$ hm.get(8) $\Rightarrow$ 3

$S \rightarrow \text{hm.get}(S)$ $\downarrow$ Null Error

if (hm.containsKey(key))

Steps → Iterate on array & build hashmap

if an ele is present in HM



```
public static void Queryfreq(int[]A, int[]Q){
```

```
    HashMap<Integer,Integer>hm=new HashMap<>();
```

```
    for(int i =0 ;i<A.length;i++){  
        int ele=A[i];
```

```
        if(hm.containsKey(ele)==true){
```

```
            int of=hm.get(ele);
```

```
            hm.put(ele,of+1);
```

```
        }else{
```

```
            hm.put(ele,1);
```

```
        }
```

```
    }
```

Tc: $O(N + Q)$

Sc: $O(N)$

```
    for(int i=0;i<Q.length;i++){
```

```
        int Queryele=Q[i];
```

```
        if(hm.containsKey(Queryele)){
```

```
            System.out.println(hm.get(Queryele));
```

```
        }else{
```

```
            System.out.println(0);
```

```
        }
```

```
    }
```

hm.getOrDefault(ele, 0)

If ele is present → hm.get(ele)

else → 0

hm.put(ele, hm.getOrDefault(ele, 0) + 1);

< Question > : Count of distinct elements

N = 5 [3 5 6 5 4] → Ans = 4

N = 3 [3 3 3] → Ans = 1

N = 5 [1 1 1 2 2] → Ans = 2

Idea → Insert all elements in hashset & hashset has unique property of just storing distinct ele.

```
HashSet<Integer> hs = new HashSet<>();  
for (int ele : A)  
|   hs.add(ele)  
|  
return hs.size();
```

Tc: $O(N)$
Sc: $O(N)$

< Question > : Given an $A[N]$ and K .
 Check if there exists a pair (i, j) such that, $A[i] + A[j] = K$ && $i \neq j$

arr $\rightarrow$

8	9	1	-2	4	5	11	-6	4
0	1	2	3	4	5	6	7	8

$K = 6 \rightarrow A[0] + A[3] \rightarrow \text{True}$

$K = 22 \rightarrow \text{False}$

$K = 8 \rightarrow A[4] + A[8] \rightarrow \text{True}$

Quiz :

3	5	1	2	1	2
0	1	2	3	4	5

$K = 7 \rightarrow A[1] + A[3] \rightarrow \text{True}$

$K = 10 \rightarrow \text{false}$



BF Idea $\rightarrow$ Get all unique pairs & check if their sum $= K$

$$\underbrace{A[i]}_x + \underbrace{A[j]}_y = K$$

$$y = K - x$$

For every $A[i]$, search if $K - A[i]$ is present

```
for (i=0; i<n; i++) {
```

```
    int x = A[i]
```

```
    int y = k - x;
```

```
    for (j=i+1; j<n; j++)
```

```
        if (A[j] == y) return true;
```

```
    }
```

```
return false;
```

TC: $O(N^2)$

SC: $O(1)$

* Idea 2 → Use HashSet



Insert all elements in HashSet & then search for x & y

$A() = \{ 8, 9, 2, -2, 4, 5, 11 \}$ $k = 9$

$x = 8, 9, 2, -2$

$y = 1, 0, 7, 11$
($k-x$)

if (hs.contains(11))



return true;

hs

8	9	2	-2
4	5	11	

Issue →

$A() = \{ 8, 9, 2, -2, 4, 5, 11 \}$ $k = 16$

$x = 8$

$y = 16-8 = 8$
($k-x$) $8 \rightarrow$ if (hs.contains(8)) → True

hs

8	9	2	-2
4	5	11	

Idea 3 → Use hashmap < Distinct ele, freq >

$A[] = \{8, 9, 2, -2, 9, 8\}$ $K = 16$

$x = 8$

$y = 8$

$(K-x)$

HM

8 → 2

9 → 2

2 → 1

-2 → 1

* check if y is present in HM

↳ if $(x \neq y) \rightarrow$ return true;

↳ if $(x = y) \rightarrow$ if $(\text{hm.get}(y) > 1)$
↳ return true;

`HashMap<I, I> hm = new HashMap<>();`

`for (int ele : A) {`

`hm.put(ele, hm.getOrDefault(ele, 0) + 1);`

`for (int i = 0; i < A.length; i++)`

`int x = A[i], y = K - x;`

`if (hm.containsKey(y))`

`if  $(x \neq y)$  return true;`

`if  $(x = y)$  {`

`if  $(\text{hm.get}(x) > 1)$  return true;`

`return false;`

TC: $O(N+N)$

SC: $O(N)$

Next class

* How to make hashtable approach works?

* Count pair sum = k